

The Ultimate Web Development Handbook



The Display Property in CSS

The `display` property controls **how an element is rendered** on the page — whether it takes up a full line, shares space with others, behaves like a container, or is completely hidden.

Understanding how display works is critical to mastering layout in CSS.

Common Display Values

1. `block`

- The element takes up the **full width** of its container.
- Starts on a **new line**.

Examples of block elements: `<div>` , `<p>` , `<h1>` – `<h6>` , `<section>` , `<article>`

```
div {  
  display: block;  
}
```

2. `inline`

- The element takes up **only as much width** as its content.
- Can appear **next to other inline elements**.
- **Cannot** set width, height, margin-top/bottom, or padding-top/bottom effectively.

Examples: `<span>` , `<a>` , `<strong>` , `<em>`

```
span {  
  display: inline;  
}
```

3. inline-block

- Behaves like `inline` but **allows width, height, margin, and padding** to be set.
- Does **not** force a line break.

```
button {  
  display: inline-block;  
  width: 150px;  
  height: 40px;  
}
```

4. none

- Completely **hides** the element from the page.
- The element is **not rendered**, and does **not take up any space**.

```
.alert {  
  display: none;  
}
```

Useful for toggling visibility dynamically (e.g., with JavaScript).

5. flex and grid (Coming Soon)

These are modern layout tools you'll learn in upcoming lessons:

- flex enables 1D flexible layouts.
 - grid enables 2D grid layouts.
-

Visual Example

```
<div style="display: inline-block; width: 100px; background: lightgray;">
  Box 1
</div>
<div style="display: inline-block; width: 100px; background: lightblue;">
  Box 2
</div>
```

These boxes appear **side by side** with a fixed width.

Changing Display in Practice

```
nav {
  display: block;
}

nav a {
  display: inline-block;
  padding: 10px;
}
```

Summary

- `block` : Full width, starts new line.
- `inline` : Sits within a line, no block features.
- `inline-block` : Inline behavior with block features.
- `none` : Removes the element completely.
- `flex` and `grid` : Modern layouts covered soon.

Positioning in CSS

CSS positioning allows you to **move elements** from their default flow and place them **precisely** where you want on the page. It's an essential part of creating modern, interactive layouts.

The `position` Property

There are five main values:

Value for Position	Description
<code>static</code>	Default. Element stays in the normal document flow
<code>relative</code>	Moves the element relative to its normal position
<code>absolute</code>	Removes from flow; positions relative to nearest positioned ancestor
<code>fixed</code>	Positions the element relative to the browser window , even on scroll
<code>sticky</code>	Behaves like <code>relative</code> , but sticks to a position while scrolling

1. `static` (Default)

Every element is positioned statically by default.

```
div {  
  position: static;  
}
```

You can't move statically positioned elements with `top` , `left` , etc.

2. `relative`

Moves the element **relative to where it would normally be**.

```
.box {  
  position: relative;  
  top: 20px;  
  left: 10px;  
}
```

It stays in the document flow, but shifts slightly.

3. `absolute`

- Removes the element from normal flow.
- Positions it **relative to the closest ancestor with `position` set** (not `static`).
- If no positioned ancestor, it uses the `<html>` element.

```
.parent {  
  position: relative;  
}  
  
.child {  
  position: absolute;  
  top: 0;
```

```
right: 0;  
}
```

The `.child` will stick to the top-right of `.parent`.

4. `fixed`

- Stays in a fixed position **relative to the viewport**.
- Does **not move** when scrolling.

```
.banner {  
  position: fixed;  
  top: 0;  
  left: 0;  
  width: 100%;  
}
```

Great for sticky headers, floating buttons, or back-to-top links.

5. `sticky`

- Acts like `relative` until a scroll threshold is reached, then behaves like `fixed`.

```
.heading {  
  position: sticky;  
  top: 0;  
  background: white;  
}
```

Sticky headers or sidebars often use this behavior.

top, right, bottom, left

These properties only work with `relative`, `absolute`, `fixed`, or `sticky`.

```
.box {  
  position: absolute;  
  top: 50px;  
  left: 100px;  
}
```

z-index

Controls the **stacking order** of overlapping elements.

```
.modal {  
  position: absolute;  
  z-index: 100;  
}
```

Higher `z-index` values appear **above** lower ones.

Summary

- Use `relative` for minor adjustments.
- Use `absolute` to fully control placement inside containers.
- Use `fixed` for elements that stay on screen while scrolling.
- Use `sticky` for scroll-based sticky behaviors.
- Always understand the **positioning context** — especially when using `absolute`.

